

OBJETIVOS

- Familiarizar o aluno com o SO MINIX e testes em ambiente de máquina virtual.
 - Compilar e modificar o kernel
- Criação de processos, IPC, gerenciamento de sinais
 - Processos órfãos e zumbis
- Gerenciamento de I/O no MINIX
 - Criar uma chamada de sistema;
 - Criar uma chamada de *kernel* e usá-la via uma chamada de sistemas (item anterior);
 - Criar um *character device*.

TAREFA 9 (Entrada/Saída): criando um controlador de dispositivo simples (*device driver*) (de 17/03/2025 a 20/03/2025)

Esta prática tem o intuito de apresentar as bases para a programação de controladores de dispositivos no Minix usando linguagem C. Um *device driver* é um programa de computador, o qual interage com componentes de hardware reais. Por exemplo, o computador possui *device drivers* para dispositivos como o monitor, disco rígido etc.

Requisitos

O desenvolvimento de um *device driver* requer habilidades de programação e conhecimento do dispositivo alvo. Nesta prática, assume-se que o aluno possui conhecimentos básicos de linguagem C e que já tenha trabalhado algum tempo com o sistema Minix. Para esta prática, será estudado um *device driver* bastante simples; o 'Olá, Mundo!'. Este *device driver* possui apenas uma tarefa: imprimir uma mensagem de olá em seu início e então terminar.

Observe que este *device driver* já existe e pode ser encontrado em `/usr/src/drivers/hello`. O resto deste documento irá assumir que o *device driver* não está lá e você irá criá-lo por conta própria.

(a) O primeiro passo é criar um diretório na árvore de diretórios do Minix para colocar o código fonte do seu *driver*. Como *root* execute:

```
# cd /usr/src/drivers
# mkdir ola
```

```
# cd ola
```

Para compilar o *device driver* é necessário criar um Makefile. Você pode usar o seguinte código em seu **Makefile**:

```
# Makefile for the hello driver.
```

```
PROG=    ola
```

```
SRCS=    ola.c
```

```
DPADD+=  ${LIBDRIVER} ${LIBSYS}
```

```
LDADD+=  -ldriver -lsys
```

```
MAN=
```

```
BINDIR?= /usr/sbin
```

```
.include <bsd.prog.mk>
```

Posteriormente, crie o arquivo `ola.c` para o código do *driver*:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <minix/syslib.h>
```

```
int main(int argc, char **argv)
```

```
{
```

```
    sef_startup();
```

```
    printf("Olá, mundo!\n");
```

```
    return EXIT_SUCCESS;
```

```
}
```

Ignore a chamada `sef_startup()` por agora. Esta será explicada posteriormente. **Teste a compilação deste código.**

```
# make clean
# make
# make install
```

Se você não recebeu quaisquer erros, é o momento de tentar executar o *driver*. Antes de fazer isso, devemos modificar o arquivo `/etc/system.conf`. Ele contém uma lista de todos os serviços de sistema (servidores e *drivers*) e suas permissões. Cada *device driver* precisa normalmente acessar um dispositivo de hardware real e usa algumas poucas funções oferecidas pelo Minix. Para testar o *driver* permissões suficientes, teste-o com a seguinte configuração no `/etc/system.conf`:

```
service ola
{
    system
        IRQCTL          # 19
        DEVIO           # 21
    ;
    ipc
        SYSTEM pm rs log tty ds vm vfs
        pci inet amddev
    ;
    uid 0;
};
```

O procedimento de iniciar (*start*), parar (*stop*) e reiniciar (*restart*) *device drivers* deve ser realizado usando o comando *service* (8). Para iniciar o programa *ola*, digite o seguinte comando:

```
# service up /usr/sbin/ola
Olá, mundo!
```

No terminal 1, aparece a mensagem **Olá, mundo!** Caso deseje verificar a mensagem usando o arquivo de mensagens do sistema (provenientes do kernel) use: **tail -f**

/var/log/messages

Para parar o *driver* use:

service down ola

OBS. Em versões mais recentes do Minix, o *Reincarnation Server* (RS) envia mensagens para a operação de início e parada do *driver*. Infelizmente, para a versão usada durante esse curso, isso não acontece.

Como o Minix opera como um *microkernel*, os *device drivers* são programas separados os quais enviam e recebem mensagens para se comunicar com outros componentes de sistema. *Device drivers*, como qualquer outro programa, podem conter *bugs* e podem sofrer falhas a qualquer momento. O RS tentará reiniciar os *device drivers* ao perceber que são abruptamente terminados pelo kernel devido a uma falha (*crash*), ou em caso de uma chamada `exit(2)` inesperada. O RS pode ser visto na lista de processos como *rs*, ao usar o comando `ps(1)`. O RS envia mensagens *keep-a-live* para cada *device driver* em execução no sistema, periodicamente, para assegurar que eles ainda estejam responsivos, e não parados em um laço infinito.

O *driver ola* responde às mensagens *keep-a-live*, de modo que o RS detecta corretamente se este ainda está em execução. Felizmente, como veremos no próximo exercício, existe uma biblioteca, a *libdriver* (ou *libchardriver* em versões mais recentes) no Minix. Esta biblioteca lida com várias tarefas comuns a todos os *character devices*. Por exemplo, desempenha a comunicação com o RS e os componentes do SO de forma transparente.

Por outro lado, uma funcionalidade que é diretamente exposta ao desenvolvedor de *device driver* é o protocolo de inicialização. Quando um novo *device driver* (ou qualquer outro serviço de sistema) é iniciado, o RS irá enviar ao *driver* uma mensagem de inicialização com informação sobre como ser inicializado adequadamente. Espera-se que o *driver* responda com um OK (inicialização completada de forma bem sucedida) ou com um erro (inicialização falhou). No primeiro caso, o RS assumirá que o novo *device driver* foi inicializado corretamente. No último caso, o RS irá imediatamente terminar o *driver* sem tentar reiniciá-lo. O usuário será informado com uma mensagem no console (console 1). Felizmente, o protocolo de inicialização é completamente ocultado no **System Event Framework (SEF)**, o qual expõe chamadas de

biblioteca as quais deixam que os desenvolvedores tratem a inicialização de forma simples e efetiva.

No início do código *main()*, o desenvolvedor precisa registrar uma ou mais chamadas de tratamento (*callbacks*) e então deixam o SEF realizar o restante pela chamada de *sef_startup()*. Esta é a razão pela qual inserimos uma chamada *sef_startup()* no código supracitado. Cada chamada de *callback* nada mais é do que uma função que recebe dados de inicialização como entrada e retorna um código de *status* para determinar o resultado do processo de inicialização. Por agora, os desenvolvedores podem registrar *callbacks* para três tipos de inicializações: *fresh* (quando um serviço é iniciado pela primeira vez), *live update* (quando o serviço é dinamicamente atualizado para uma nova versão), *restart* (quando um serviço é reiniciado após uma falha (*crash*) ou um reinício controlado).

(b) Criação de um *device* /dev/ola

Neste exercício, você irá estender o *driver ola* e re-implementá-lo usando a *libdriver* (*libchardriver* na versão 3.2.0). Ao invés de apenas imprimir uma mensagem 'Olá' no início, agora iremos usar um arquivo de dispositivo /dev/ola para ler a mensagem 'Olá, mundo!'. Cada *driver* de caractere e bloco é associado com um número de dispositivo maior ordem (*major*). Assim, é necessário escolher um número de dispositivo *major* livre para o dispositivo. O número *major* usado será o 17 para ID do dispositivo. Para começar, criamos o arquivo de dispositivo:

```
# mknod /dev/ola c 17 0
```

Explique exatamente o que este comando realiza em seu relatório.

Agora, reutilizaremos o Makefile da letra (a). Crie um arquivo de cabeçalho *ola.h*:

```
#ifndef __OLA_H
#define __OLA_H

/** A mensagem 'Ola, mundo!'. */
#define MENSAGEM_OLA "Ola, mundo!\n"

#endif /* __OLA_H */
```

Este arquivo possui uma macro de pré-processamento para a mensagem. Agora coloque o arquivo no mesmo diretório de *ola.c* e, acrescente-o dentre os seus arquivos de cabeçalho. Nosso código C, *ola.c* será modificado como segue (copie e cole */usr/src/drivers/hello/hello.c* e modifique):

```
#include <minix/drivers.h>
#include <minix/driver.h>
#include <stdio.h>
#include <stdlib.h>
#include <minix/ds.h>
#include "ola.h"

/*
 * Prototipos de funcao para o driver ola.
 */
FORWARD _PROTOTYPE( char * ola_name,    (void) );
FORWARD _PROTOTYPE( int ola_open, (struct driver *d, message *m) );
FORWARD _PROTOTYPE( int ola_close, (struct driver *d, message
*m) );
FORWARD _PROTOTYPE( struct device * ola_prepare, (int device) );
FORWARD _PROTOTYPE( int ola_transfer, (    int procnr,
                                           int opcode,
                                           u64_t position,
                                           iovec_t *iov,
                                           unsigned nr_req) );
FORWARD _PROTOTYPE( void ola_geometry, (struct partition
*entry) );

/* SEF funcoes e variaveis. */
FORWARD _PROTOTYPE( void sef_local_startup, (void) );
FORWARD _PROTOTYPE( int sef_cb_init, (int type, sef_init_info_t
*info) );
FORWARD _PROTOTYPE( int sef_cb_lu_state_save, (int) );
FORWARD _PROTOTYPE( int lu_state_restore, (void) );
```

```

/* Pontos de entrada para o driver ola. */
PRIVATE struct driver ola_tab =
{
    ola_name,
    ola_open,
    ola_close,
    nop_ioctl,
    ola_prepare,
    ola_transfer,
    nop_cleanup,
    ola_geometry,
    nop_alarm,
    nop_cancel,
    nop_select,
    nop_ioctl,
    do_nop,
};

/** Representa o dispositivo /dev/ola */
PRIVATE struct device ola_device;

/** Variavel de estado para contar o numero de vezes que o
dispositivo foi aberto. */
PRIVATE int open_counter;

PRIVATE char * ola_name(void)
{
    printf("ola_name()\n");
    return "ola";
}

```

```

PRIVATE int ola_open(d, m)
    struct driver *d;
    message *m;
{
    printf("ola_open(). Chamou %d vez(es).\n", ++open_counter);
    return OK;
}

PRIVATE int ola_close(d, m)
    struct driver *d;
    message *m;
{
    printf("ola_close()\n");
    return OK;
}

PRIVATE struct device * ola_prepare(dev)
    int dev;
{
    ola_device.dv_base.lo = 0;
    ola_device.dv_base.hi = 0;
    ola_device.dv_size.lo = strlen(MENSAGEM_OLA);
    ola_device.dv_size.hi = 0;
    return &ola_device;
}

PRIVATE int ola_transfer(proc_nr, opcode, position, iov, nr_req)
    int proc_nr;
    int opcode;
    u64_t position;
    iovec_t *iov;
    unsigned nr_req;

```



```

{
    int bytes, ret;

    printf("ola_transfer()\n");

    bytes = strlen(MENSAGEM_OLA) - position.lo < iov->iov_size ?
        strlen(MENSAGEM_OLA) - position.lo : iov->iov_size;

    if (bytes <= 0)
    {
        return OK;
    }
    switch (opcode)
    {
        case DEV_GATHER_S:
            ret = sys_safecopyto(proc_nr, iov->iov_addr, 0,
                                (vir_bytes) (MENSAGEM_OLA +
position.lo), bytes, D);
            iov->iov_size -= bytes;
            break;

            default:
                return EINVAL;
    }
    return ret;
}

```

```

PRIVATE void ola_geometry(entry)
    struct partition *entry;
{
    printf("ola_geometry()\n");
    entry->cylinders = 0;
}

```

```

    entry->heads      = 0;
    entry->sectors     = 0;
}

PRIVATE int sef_cb_lu_state_save(int state) {
/* Salva o estado. */
    ds_publish_u32("open_counter", open_counter, DSF_OVERWRITE);

    return OK;
}

PRIVATE int lu_state_restore() {
/* Restaura o estado. */
    u32_t value;

    ds_retrieve_u32("open_counter", &value);
    ds_delete_u32("open_counter");
    open_counter = (int) value;

    return OK;
}

PRIVATE void sef_local_startup()
{
    /*
     * Registra chamadas de callback init. Usa a mesma funcao para
    todos os tipos de evento
     */
    sef_setcb_init_fresh(sef_cb_init);
    sef_setcb_init_lu(sef_cb_init);
    sef_setcb_init_restart(sef_cb_init);

```

```

/*
 * Registra chamadas de live update.
 */
/* - Agree to update immediately when LU is requested in a
valid state. */
sef_setcb_lu_prepare(sef_cb_lu_prepare_always_ready);
/* - Support live update starting from any standard state. */

sef_setcb_lu_state_isvalid(sef_cb_lu_state_isvalid_standard);
/* - Register a custom routine to save the state. */
sef_setcb_lu_state_save(sef_cb_lu_state_save);

/* Let SEF perform startup. */
sef_startup();
}

PRIVATE int sef_cb_init(int type, sef_init_info_t *info)
{
/* Inicia o driver alo. */
int do_announce_driver = TRUE;

open_counter = 0;
switch(type) {
    case SEF_INIT_FRESH:
        printf("%s", MENSAGEM_OLA);
        break;

    case SEF_INIT_LU:
        /* Restore the state. */
        lu_state_restore();
        do_announce_driver = FALSE;

```

```

        printf("%sEi,    Eu    sou    uma    nova    versao!\n",
MENSAGEM_OLA);
        break;

        case SEF_INIT_RESTART:
            printf("%sEi,    acabo    de    ser    reiniciado!\n",
MENSAGEM_OLA);
            break;
    }

    /* Anuncia que esta online quando necessario. */
    if (do_announce_driver) {
        driver_announce();
    }

    /* Inicializacao bem sucedida. */
    return OK;
}

PUBLIC int main(int argc, char **argv)
{
    /*
     * Realiza inicializacao.
     */
    sef_local_startup();

    /*
     * Executa o laco principal.
     */
    driver_task(&ola_tab, DRIVER_STD);
    return OK;
}

```

Agora, vamos entender o que o código acima realiza. Primeiro, ele possui várias linhas de `#include` para os protótipos e funções necessários, usados no programa. Posteriormente, declara-se uma *struct driver*. Esta estrutura é preenchida com funções de *callback* as quais serão invocadas pelo *libdriver* (ou *libchardriver* na versão $\geq 3.2.0$) em tempo de execução. Ela possui funções de *callback* para realizar operações de *open*, *read*, *write* e *close* no *device driver*. A maior parte da ação ocorre na função *ola_transfer*. Ela é usada pela função *sys_safecopyto* para copiar a *string* `MENSAGEM_OLA` do programa *device driver* para o programa de usuário lendo o `/dev/ola`. O sistema operacional irá então, por parte do *device driver*, tomar conta de copiar os bytes entre os dois programas adequadamente. Um vetor de entrada/saída *iov* é usado pela função *sys_safecopyto* para descrever o endereço de memória para armazenar bytes e o número de bytes requisitados pela aplicação do usuário.

Na função *main()*, existem apenas duas chamadas simples. *sef_local_startup()* é chamada no início para registrar as chamadas de *callback* SEF e então completar a inicialização. Em *sef_local_startup()* a mesma função *sef_cb_init_fresh()* é registrada como uma função de *callback* para todos os tipos de inicialização suportados. Isto significa que o código de inicialização executado quando o *driver* inicia será sempre o mesmo independente do *device driver* iniciado, após um *live update*, ou após um reinício.

No *sef_local_startup()*, também registramos *callbacks* para eventos *live update*. Um evento *live update* é disparado pelo RS quando uma nova versão de um serviço de sistema está disponível. Neste ponto, o RS envia uma mensagem para a versão mais antiga do serviço pedindo para se preparar para um estado particular como requerido pela atualização (*update*). O serviço irá decidir se irá aceitar o estado de *update* solicitado ou rejeitar o *update*. No primeiro caso, o serviço se compromete a se preparar para a atualização e responde ao RS quando o estado alvo foi alcançado. Somente neste ponto, o RS instala a atualização e a nova versão pode ser iniciada a partir do estado onde a versão mais antiga parou. Felizmente, o protocolo de atualização (*live update*) está escondido do SEF e o desenvolvedor de *driver* não precisa se preocupar com todos os detalhes. Para suportar o *live update* para um serviço do sistema, o desenvolvedor deve apenas registrar *callbacks*, para deixar o SEF saber o que fazer quando uma requisição de *live update* chega. Os tipos de chamada de *callback* mais importantes para um *live update* são *state_isvalid* e *prepare*. Uma chamada de *callback state_isvalid* deve ser registrada para informar o SEF quais estados de *update* devem ser aceitos. Uma chamada de *callback prepare*, por outro lado, é chamada toda vez

que um serviço de sistema é recebe uma chance de se preparar para a atualização (após uma solicitação de *update* ser aceita). A chamada de *callback* deve pegar o estado do alvo como entrada e retornar um OK somente se o serviço estiver pronto para a atualização solicitada.

O *driver ola* é completamente *stateless* e pode suportar *live update* começando a partir de qualquer estado e deixa o RS realizar o *update* na primeira chance possível. Para completar, registramos duas chamadas de *callback* pré-definidas para os tipos *state_isvalid* e *prepare*. Estas são as funções de biblioteca implementadas dentro do SEF *sef_cb_lu_state_isvalid_standard* e *sef_cb_lu_prepare_always_ready*, respectivamente. A chamada de *callback* *sef_cb_lu_state_isvalid_standard* aceita qualquer padrão *live update* e a *callback* *sef_cb_lu_prepare_always_ready* informa que o serviço está pronto e atualizado. Quando o *device driver* não é *stateless*, o desenvolvedor precisa fornecer as implementações apropriadas para estes dois tipos de *callback*, para suportar os estados padrão e possivelmente alguns outros estados particulares definidos pelo *driver*. Os estados padrão definidos são SEF_LU_STATE_WORK_FREE (o serviço não está fazendo nenhum trabalho), SEF_LU_STATE_REQUEST_FREE (o serviço não possui requisição pendente), SEF_LU_STATE_PROTOCOL_FREE (o serviço não está envolvido em um protocolo).

Finalmente, a função *main()* executa *driver_task()*(ou *chardriver_task()*) para deixar o *libdriver* (ou *libchardriver*) começar a processar requisições de usuário e invocar as funções de *callback* do *driver ola*.

Para finalizar, compile o programa novamente usando os mesmos comandos apresentados no item (a), e inicie o driver com so comando *service*. Acrescente '*-dev /dev/ola*') para indicar que o *driver* iniciado no momento é responsável pelo dispositivo identificado por */dev/ola* – no caso, *major 17*, o qual foi escolhido anteriormente.

```
# service up /usr/sbin/ola -dev /dev/ola
Ola, mundo!
```

Perceba que não há mais reinício do RS (múltiplas ocorrências da mensagem). Agora tente verificar se é possível ler a mensagem a partir do arquivo de dispositivo. É possível fazer isso, simplesmente usando o comando *cat(1)*.

```
# cat /dev/ola
ola_open()
```

```
ola_transfer()
ola_transfer()
ola_close()
Ola, mundo!
#
```

Se você receber a mensagem acima, o *driver ola* funcionou corretamente.

Agora vamos realizar o reinício do *driver* com o comando *service(8)* para simular uma falha:

```
# service refresh ola
Ei, acabei de ser reiniciado!
# cat /dev/ola
ola_open()
ola_transfer()
ola_transfer()
ola_close()
Ola, mundo!
#
```

Finalmente, teste um *live update*. Faça a seguinte mudança no arquivo de cabeçalho *ola.h*:

```
#ifndef __OLA_H
#define __OLA_H

/** A mensagem 'Ola, mundo!'. */
#define MENSAGEM_OLA "Ola, novo mundo!\n"

#endif /* __OLA_H */
```

Agora recompile o *driver* e atualize-o com o comando *service(8)*:

```
# make clean install
# service update /usr/sbin/ola -state 1
Ola, novo mundo!
Ei, eu sou uma nova versao!
```

Com isso, se solicita uma atualização de estado onde nenhum trabalho esteja em progresso (i.e. SEF_LU_STATE_WORK_FREE=1 – esse é o padrão)

Executando o comando *cat*:

```
# cat /dev/ola
ola_open()
ola_transfer()
ola_transfer()
ola_close()
Ola, novo mundo!
#
```

Como esperado, o *driver* é atualizado ao vivo, e a nova versão é imediatamente iniciada.